

<<코드 리뷰>>

```
#include <stdio.h>
#include <sys/socket.h>
#include <unistd.h>
#include <stdlib.h>
#include <netinet/in.h>
#include <string.h>
#include <pthread.h>
```

- `stdio.h`: 표준 입출력 함수 사용
- `sys/socket.h`: 소켓 프로그래밍에 필요한 자료구조와 함수 선언
- `unistd.h`: UNIX 표준 시스템 호출 사용
- `stdlib.h`: 표준 라이브러리 함수 사용
- `netinet/in.h`: 인터넷 주소 구조체 정의
- `string.h`: 문자열 처리 함수 사용
- `pthread.h`: POSIX 스레드 사용

```
while (1)
{
    printf("\n++++++ Waiting for new connection ++++++\n\n");

    if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&address_len)) < 0)
    {
        perror("In accept");
        exit(EXIT_FAILURE);
    }

    pthread_t thread_id;
    int *new_sock = malloc(sizeof(int));
    *new_sock = new_socket;

    if (pthread_create(&thread_id, NULL, handle_connection, (void *)new_sock) != 0)
    {
        perror("could not create thread");
        exit(EXIT_FAILURE);
    }

    printf("Handler assigned\n");
}
return 0;
}
```

무한 루프를 돌면서 클라이언트의 연결을 기다림 새로운 연결이 들어오면, 이를 처리하기 위해 새로운 스레드를 생성.

```

void *handle_connection(void *socket_desc)
{
    int sock = *(int *)socket_desc;
    long valread;
    char *hello = "HTTP/1.1 200 OK\nContent-Type: text/plain"
                  "Content-Length: 20\n\nMy first web server!";
    char buffer[30000] = {0};

    valread = read(sock, buffer, 30000);
    printf("%s\n", buffer);

    write(sock, hello, strlen(hello));
    printf("-----Hello message sent-----\n");

    close(sock);
    free(socket_desc);

    return NULL;
}

```

Pthread_create()를 통해 각 thread마다 connection을 클라이언트와 가짐

<<특이사항>>

Pthread_join()함수를 따로 사용하지는 않았는데 그 이유는 http request의 개수가 많지가 않기도 하고 무엇보다 while(1)문을 통해 간접적으로 동기화가 수행되는 경우에 속한다고 판단 할 수 있음